

ADC Acquisition

ADC Acquisition

1. Learning Objectives
 - ADC Principle
 - ADC Basic Parameters
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. ADC Parameter Configuration
 3. ADC Acquisition Channel Configuration
 4. Write Program
 5. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Understand basic ADC knowledge.
2. Learn the basic use of ADC and display the acquired values on the serial assistant.

ADC Principle

ADC (analog to digital converter) is used to convert analog signals (such as voltage) into digital signals. Analog signals change continuously, while digital signals are discrete binary numbers. ADC converts analog signals to digital data through sampling and quantization, so that processors or microcontrollers can perform subsequent processing. According to conversion principles, they are mainly divided into three types: successive approximation, dual-slope, and voltage-to-frequency conversion.

MSPM0G3519 uses successive approximation (SAR) ADC, which is a common ADC working principle. Its basic idea is to gradually approach the digital representation of the input signal by comparing the size relationship between the analog signal and the reference voltage. In the successive approximation ADC, the input signal and reference voltage are compared through a differential amplifier to generate a differential voltage. Then, this differential voltage is input to a gradually approaching digital quantizer, which compares it with a series of reference voltages at each approximation stage. At each approximation stage, the quantizer compares the input signal with an intermediate voltage point and selects a higher or lower reference voltage as the reference for the next approximation stage based on the comparison result. This process continues until the quantizer finally approaches a digital output value.

ADC Basic Parameters

1. **Resolution:**
 - Resolution represents the accuracy of ADC converter output, usually measured in bits (bit), such as 8-bit, 10-bit, 12-bit, etc. Higher resolution means the ADC can represent more discrete digital values, thereby providing higher accuracy.

- **MSPM0G3519** supports 8-bit, 10-bit, and 12-bit resolution. Users can select the appropriate resolution for configuration based on their needs.

2. Sample Rate:

- Sample rate (also called conversion rate) represents the rate at which ADC samples the analog input signal, usually expressed in samples per second (SPS). It indicates how many analog-to-digital conversions the ADC can perform per second.
- **MSPM0G3519** has a sample rate of 4Msps (4 million samples per second), suitable for high-frequency signal acquisition and real-time data processing.

3. Voltage Reference:

- The ADC voltage reference is a reference voltage used to compare with the analog input signal to finally achieve conversion from analog signal to digital signal. The accuracy and stability of the voltage reference are crucial to the ADC conversion accuracy.
- MSPM0G3519 supports three voltage reference configurations:
 - Internally configurable reference voltage: 1.4V and 2.5V dedicated ADC reference voltage (VREF).
 - MCU power supply voltage (VDD) as the reference voltage.
 - External reference voltage provided through VREF+ and VREF- pins. If the voltage reference is not configured, the MCU power supply voltage (VDD) is used as the reference voltage by default.

4. Sampling Range:

- Sampling range represents the voltage range of analog input signals that ADC can acquire, which is closely related to the reference voltage setting. The range is as follows: $V_{REF-} \leq \text{ADC} \leq V_{REF+}$
 - **VREF-**: The set reference voltage negative terminal, usually 0V.
 - **VREF+**: The set reference voltage positive terminal, determined by software configuration.

These parameters together determine the ADC performance, including its accuracy, response speed, and input voltage range.

2. Hardware Setup

MSPM0G3519 uses a 12-bit successive approximation ADC. It has 17 multiplexed channels for conversion. The 17 external channels each correspond to a pin on the microcontroller. This pin is not fixed. For details, please refer to the pin diagram or data manual.

									A0_1	(Non-IOMUX 1) 0	A	
									COMP0_IN0+	(Non-IOMUX 2) 0	A	
31	B2	G2	47	47	31	79	99	PA27 PINCM60 0x404280ec	PA27	1	IO	SDIO (standard)
									UART3_RX	2	I	
									SPI1_CS1	3	IO	
									TIM8_C1	4	IO	
									TIMA_FAL2	5	I	
									CLK_OUT	6	O	
									RTC_OUT	8	O	
									COMP0_OUT	9	O	
									CAN0_RX	10	I	
									TIM7_C1	11	IO	
									A0_0	(Non-IOMUX 1) 0	A	
									COMP0_IN0-	(Non-IOMUX 2) 0	A	

Table 6-3. Analog to Digital Converter (ADC) Signal Descriptions

SIGNAL NAME	PIN TYPE	DESCRIPTION	RHB PIN	ZAW PIN	YCJ PIN	RGZ PIN	PT PIN	PM PIN	PN PIN	PZ PIN
A0_0	A	ADC0 analog input channel 0	31	B2	G2	47	47	31	79	99
A0_1	A	ADC0 analog input channel 1	30	C2	G3	46	46	30	78	98
A0_2	A	ADC0 analog input channel 2	29	D1	F3	45	45	26	74	94
A0_3	A	ADC0 analog input channel 3	28	E2	F4	44	44	25	73	93
A0_4	A	ADC0 analog input channel 4		C1				27	75	95

Various channel A/D conversions can be configured in **single or sequence conversion** mode.

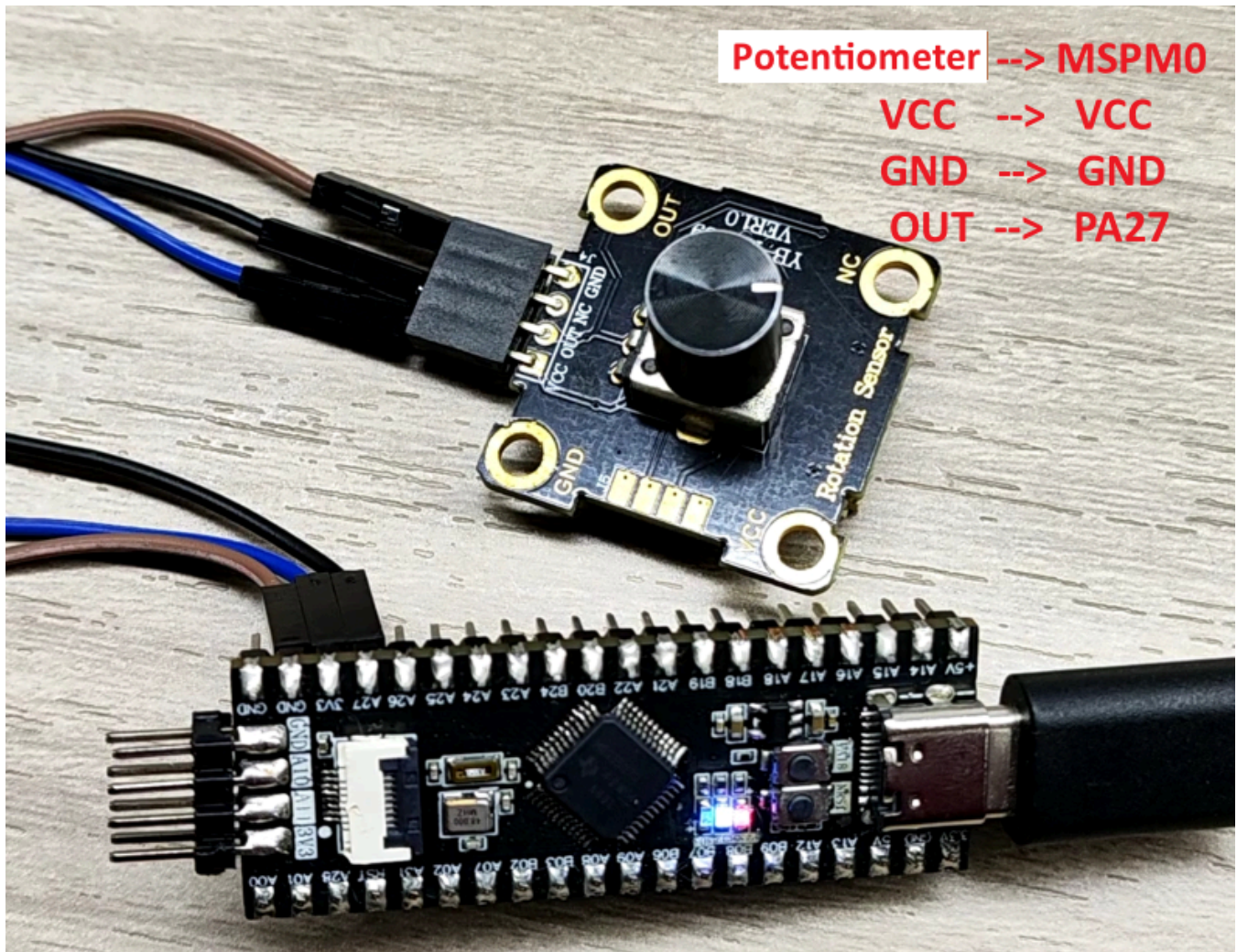
Single Conversion Mode: After each ADC conversion is completed once, the ADC automatically stops and stores the result in the ADC data register.

Repeated Single Conversion Mode: When the ADC completes one conversion, it automatically starts another conversion, continuously performing conversions until stopped by an external trigger or software trigger.

Multi-channel Sequential Single Conversion Mode: Used for sequential conversion of multiple input channels. In this mode, the ADC samples and converts multiple channels sequentially according to the configured channel acquisition order.

Multi-channel Sequential Repeated Conversion Mode: Used for sequential repeated conversion of multiple input channels. In this mode, the ADC performs repeated sampling and conversion of multiple channels sequentially according to the configured channel acquisition order.

This case acquires the PA27 pin voltage through ADC. The potentiometer module and Dupont wires needed for the ADC acquisition experiment need to be purchased separately. You can also use other input voltage methods.

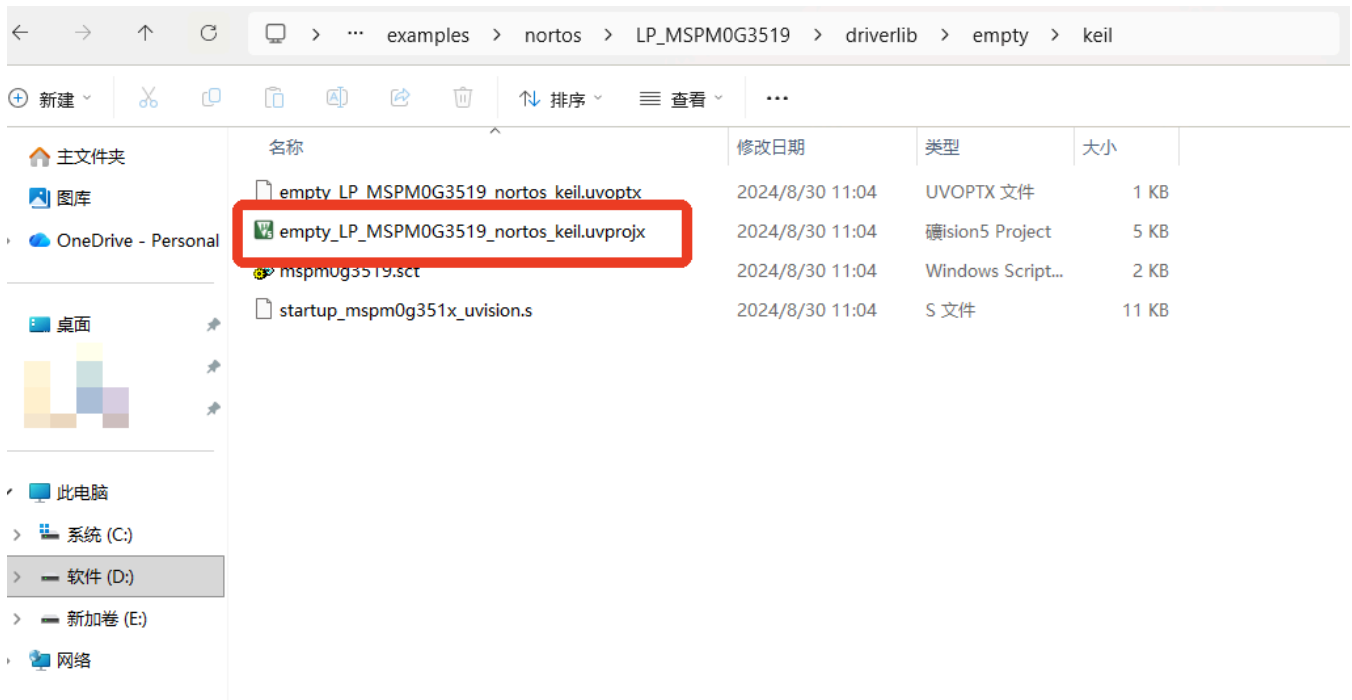


3. Experiment Steps

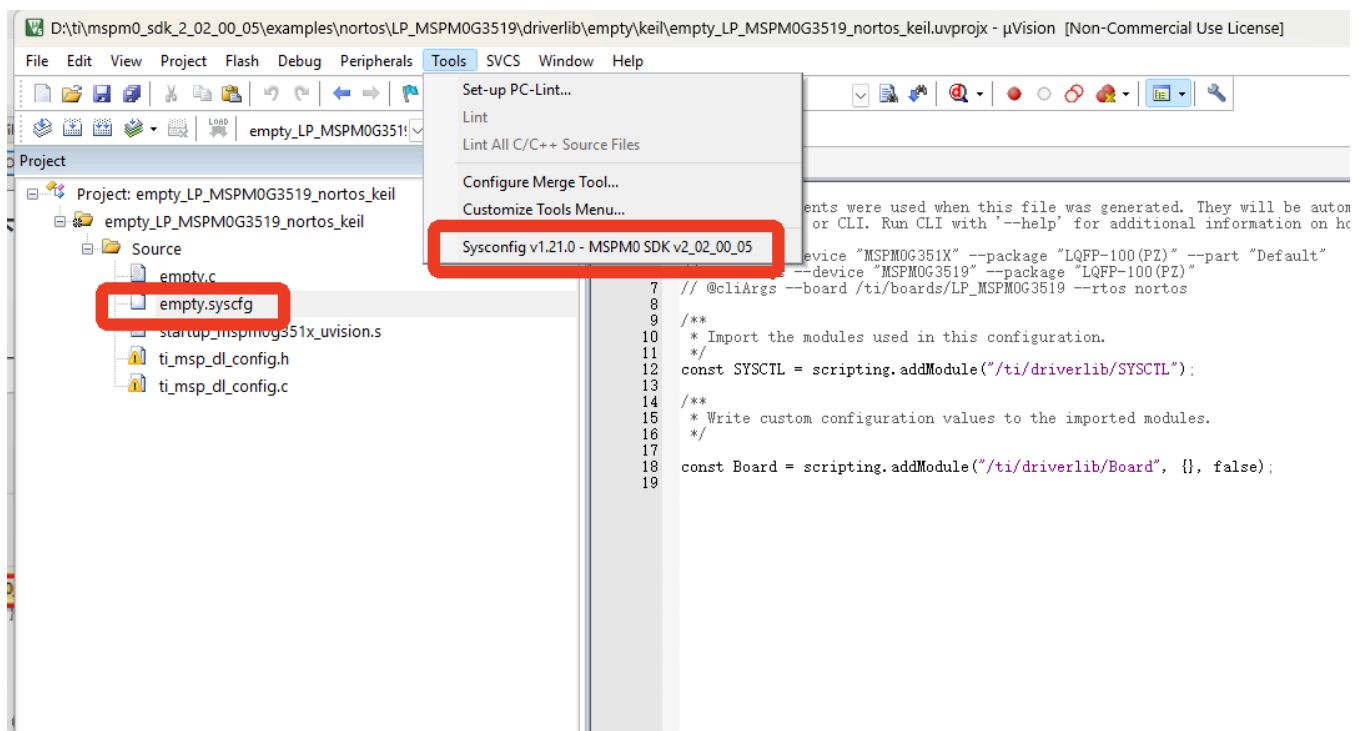
This case uses PA27 pin voltage acquisition as the experimental case. Through ADC acquisition of its voltage changes, implement serial port printing of voltage status.

1. Open SYSCONFIG Configuration Tool

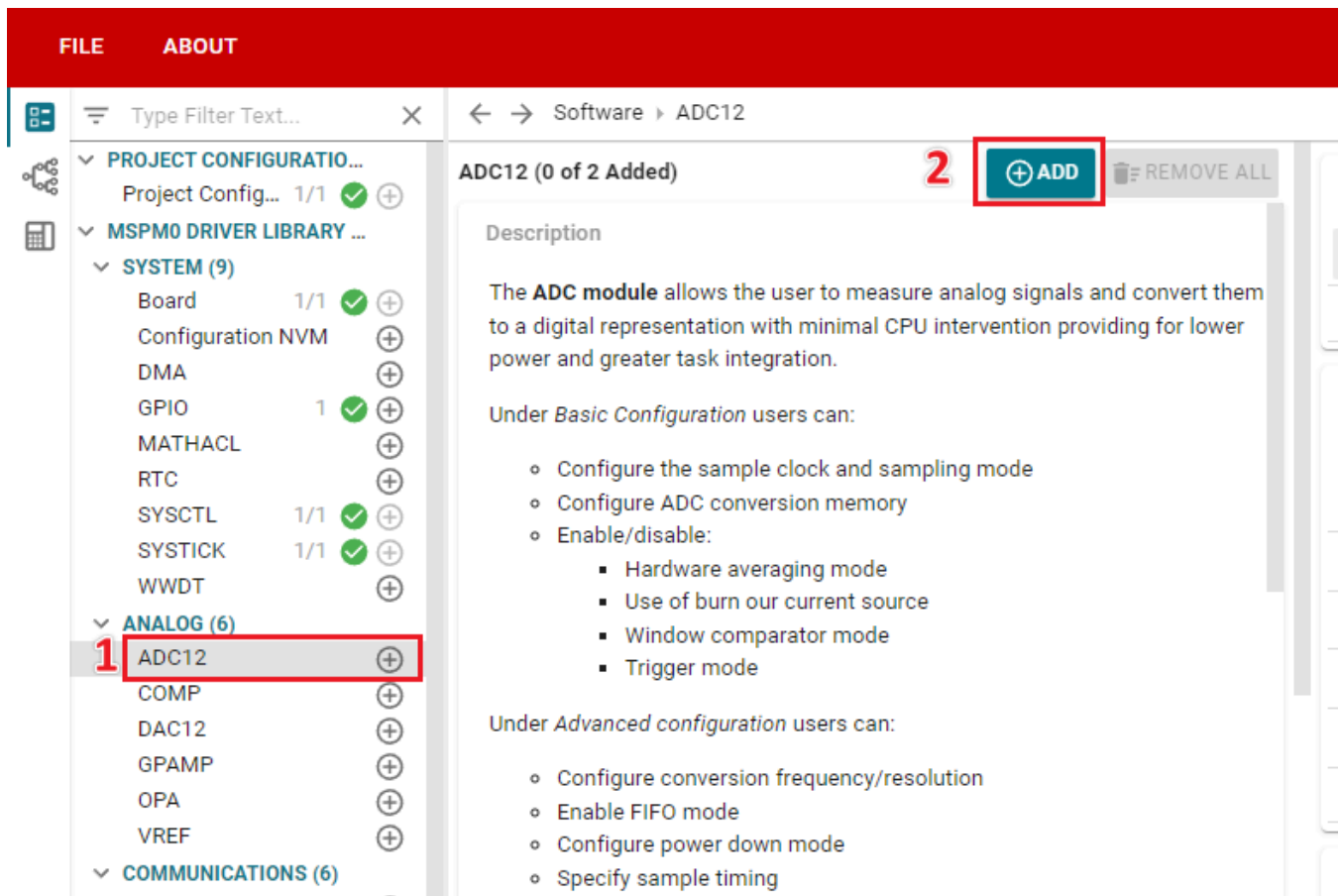
Open the empty project from the SDK in KEIL.



After selecting and opening it, open the empty.syscfg file in the KEIL interface. **With the empty.syscfg file open**, then open the SYSCONFIG GUI interface from the Tools menu.



In SYSCONFIG, select MCU peripherals on the left, find the ADC12 option and click to enter. Click ADD in ADC12 to add an ADC peripheral.



2. ADC Parameter Configuration

For serial configuration, please refer to the **Serial Communication** course. It will not be repeated here.

The configuration here uses 32MHz ADC frequency, 4MHz sample rate, single repeated conversion mode, software-triggered ADC start sampling, and binary right-aligned data format.

Parameter description:

ADC Clock Frequency: Shows the current ADC clock frequency.

Calculated Sample clock Frequency: Shows the sample frequency after division.

Conversion Starting Address: Conversion starting address. Configure to start sampling from the 0th (related to the storage register later).

Enable Repeat Mode: Whether to enable repeat conversion. Checked here, enabled.

Sampling Mode: Sampling mode. Set to `Auto`, automatic.

Trigger Source: ADC trigger mode. Configure as `Software` software trigger mode.

Conversion Data Format: ADC data conversion format. Configure as `Binary unsigned, right aligned` binary format right-aligned.

MSPM0G3519's ADC supports multiple data alignment methods to adapt to different application scenarios. Common data alignment methods include **left-aligned** and **right-aligned**.

- **Right-aligned mode:** In right-aligned mode, ADC data is right-aligned to the lowest bit after conversion, with insufficient bits filled with 0 in the high bits. Right-aligned mode provides better dynamic range without precision loss.

Rule group data

0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
---	---	---	---	-----	-----	----	----	----	----	----	----	----	----	----	----

Injection group data

Sign	Sign	Sign	Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
------	------	------	------	-----	-----	----	----	----	----	----	----	----	----	----	----

DAL=0

- **Left-aligned mode:** In left-aligned mode, ADC data is left-aligned to the highest bit, with insufficient bits filled with 0 in the low bits. Left-aligned mode can improve resolution but will cause reduced dynamic range.

Rule group data

0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
---	---	---	---	-----	-----	----	----	----	----	----	----	----	----	----	----

Injection group data

Sign	Sign	Sign	Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
------	------	------	------	-----	-----	----	----	----	----	----	----	----	----	----	----

DAL=0

We configured as right-aligned mode, so we can directly perform operations on the converted data.

3. ADC Acquisition Channel Configuration

ADC Conversion Memory Configurations

Active Memory Control Blocks

ADC Conversion Memory 0

ADC Conversion Memory 0 Configuration

Name

ADC_CH0

Input Channel

Channel 0

Device Pin Name

PA27

Reference Voltage

VDDA

VDDA

3.30 V

Sample Period Source

Sampling Timer 0

ADC Conversion Period

10.00 ms

Optional Configuration

									A0_1	(Non-IOMUX 1) 0	A	
									COMP0_IN0+	(Non-IOMUX 2) 0	A	
									PA27	1	IO	
									UART3_RX	2	I	
									SPI1_CS1	3	IO	
									TIMG8_C1	4	IO	
									TIMA_FAL2	5	I	
									CLK_OUT	6	O	
									RTC_OUT	8	O	
									COMP0_OUT	9	O	
									CAN0_RX	10	I	
									TIMG7_C1	11	IO	
									A0_0	(Non-IOMUX 1) 0	A	
									COMP0_IN0-	(Non-IOMUX 2) 0	A	
31	B2	G2	47	47	31	79	99	PA27 PINCM60 0x404280ec				SDIO (standard)

Table 6-3. Analog to Digital Converter (ADC) Signal Descriptions

SIGNAL NAME	PIN TYPE	DESCRIPTION	RHB PIN	ZAW PIN	YCJ PIN	RGZ PIN	PT PIN	PM PIN	PN PIN	PZ PIN
A0_0	A	ADC0 analog input channel 0	31	B2	G2	47	47	31	79	99
A0_1	A	ADC0 analog input channel 1	30	C2	G3	46	46	30	78	98
A0_2	A	ADC0 analog input channel 2	29	D1	F3	45	45	26	74	94
A0_3	A	ADC0 analog input channel 3	28	E2	F4	44	44	25	73	93
A0_4	A	ADC0 analog input channel 4		C1				27	75	95

- Parameter description:
- Active Memory Control Blocks:** ADC data storage address. Store ADC conversion result in register 0.
 - Input Channel:** Input channel. Select channel 0 (each channel corresponds to a different pin).
 - Device Pin Name:** Pins automatically selected based on channel. Channel 0's pin is PA27.
 - Reference Voltage:** Voltage reference. Configure to use MCU power VDDA.
 - ADC Conversion Period:** Shows the time for ADC to convert a single data.

Advanced Configuration

99.98 Hz

When Power Down Mode is set to Auto, ADC wakeup time may need to be considered in each sample window. Refer to the device specific data sheet for specifications on the ADC Wakeup Time.

Total Conversion Frequency

Conversion Resolution

12-bits

Enable FIFO Mode

☐

Power Down Mode

Auto

Desired Sample Time 0

10 ms

Actual Sample Time 0

10.00 ms

Desired Sample Time 1

0 ms

Actual Sample Time 1

0.00 s

Interrupt Configuration

Enable Interrupts

MEM0 result loaded interrupt

Interrupt Priority

Default

Parameter description:

Conversion Resolution: Resolution configured as 12-bits.

Desired Sample Time 0: Sample time set to 10ms.

Enable interrupts: Enable ADC acquisition complete interrupt. When ADC acquisition completes, it triggers an interrupt, and we perform data processing.

PinMux

Peripheral and Pin Configuration

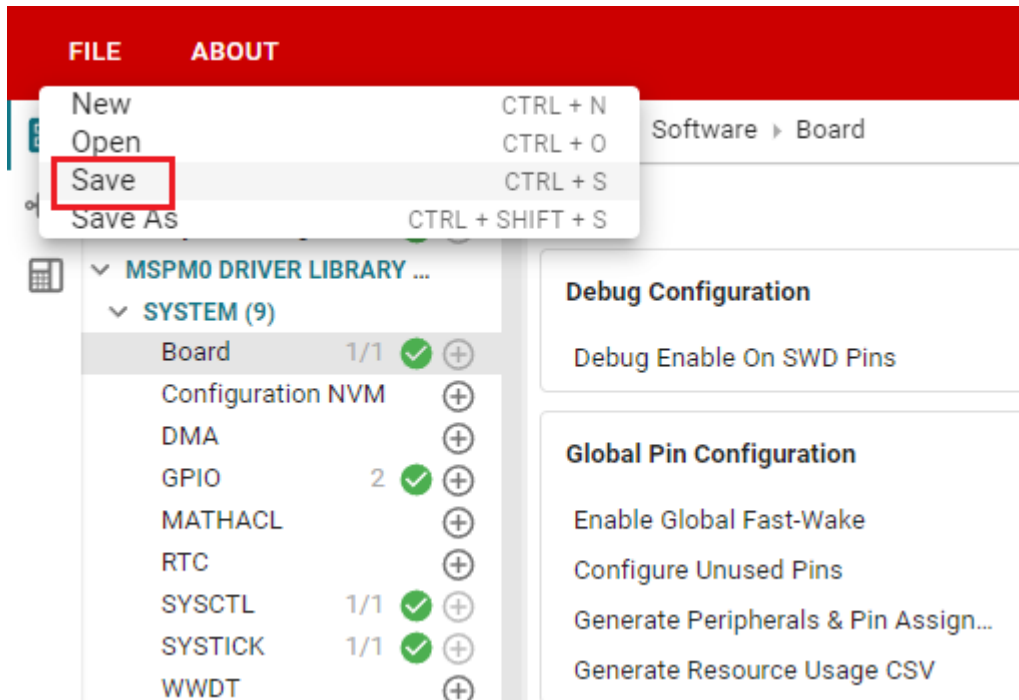
ADC12 Peripheral

ADC0

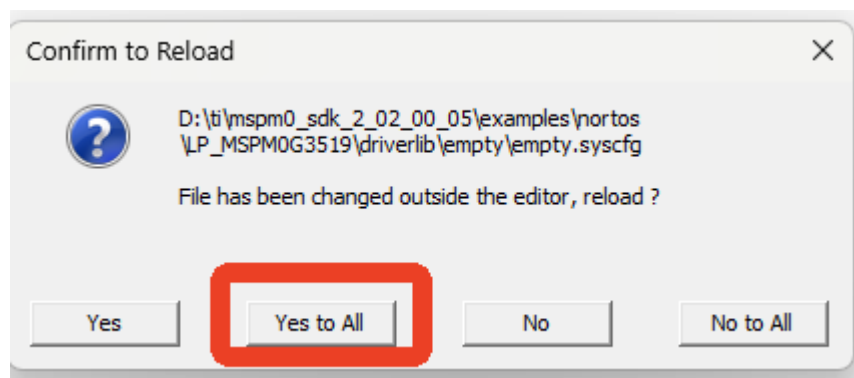
ADC12 Channel 0 Pin

PA27/31

Click SAVE to save the SYSCONFIG configuration, then close SYSCONFIG and return to KEIL.



Select **Yes** to **All** in the popup.



We also need to confirm whether `ti_msp_dl_config.c` and `ti_msp_dl_config.h` files are updated. Simply compile, and the compilation will automatically update to KEIL. If not updated, we can also copy the file content from `SYSCONFIG`.

4. Write Program

In the `empty.c` file, write the following code

```
#include "ti_msp_dl_config.h"
#include "stdio.h"

volatile unsigned int delay_times = 0;
volatile bool gCheckADC;          //ADC acquisition success flag

void delay_ms(unsigned int ms); //Precise millisecond delay with tick timer
unsigned int adc_getValue(void); //Read ADC data

/*****Serial port
redirection*****/
#if !defined(__MICROLIB)
```

```

//If you don't use the micro library, you need to add the following function
#if (__ARMCLIB_VERSION <= 6000000)
//If the compiler is AC5, define the following structure
struct __FILE
{
    int handle;
};
#endif

FILE __stdout;

// define _sys_exit() to avoid using semihost mode
void _sys_exit(int x)
{
    x = x;
}
#endif
//printf function redefinition
int fputc(int ch, FILE *stream)
{
    // wait when serial port 0 is busy, and send the incoming characters when it is not
    busy
    while( DL_UART_isBusy(UART_0_INST) == true );

    DL_UART_Main_transmitData(UART_0_INST, ch);

    return ch;
}
/*****/

int main(void)
{
    unsigned int adc_value = 0;
    unsigned int voltage_value = 0;

    SYSCFG_DL_init();

    // Clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    // Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
    // Enable ADC interrupt
    NVIC_EnableIRQ(ADC_VOLTAGE_INST_INT_IRQN);

    printf("adc Demo start\r\n");
    while (1)
    {
        // Get ADC data
        adc_value = adc_getValue();
        printf("adc value:%d\r\n", adc_value);

        // Convert the data collected by ADC into voltage
        voltage_value = (int)((adc_value/4095.0*3.3)*100);
    }
}

```

```

        printf("voltage value:%d.%d%d\r\n",
            voltage_value/100,
            voltage_value/10%10,
            voltage_value%10 );

        delay_ms(1000);
    }
}

// Delay function
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

// Read ADC data
unsigned int adc_getValue(void)
{
    unsigned int gAdcResult = 0;

    // Software triggers ADC to start conversion
    DL_ADC12_startConversion(ADC_VOLTAGE_INST);
    // If the current state is converting, wait for the conversion to end
    while (false == gCheckADC) {
        __WFE();
    }
    // Get data
    gAdcResult = DL_ADC12_getMemResult(ADC_VOLTAGE_INST, ADC_VOLTAGE_ADCMEM_ADC_CH0);

    // Clear flag
    gCheckADC = false;

    return gAdcResult;
}

// Tick timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

// ADC interrupt service function
void ADC_VOLTAGE_INST_IRQHandler(void)
{
    // Query and clear ADC interrupt
    switch (DL_ADC12_getPendingInterrupt(ADC_VOLTAGE_INST))
    {
        // Check whether data collection is completed
        case DL_ADC12_IIDX_MEM0_RESULT_LOADED:
    }
}

```

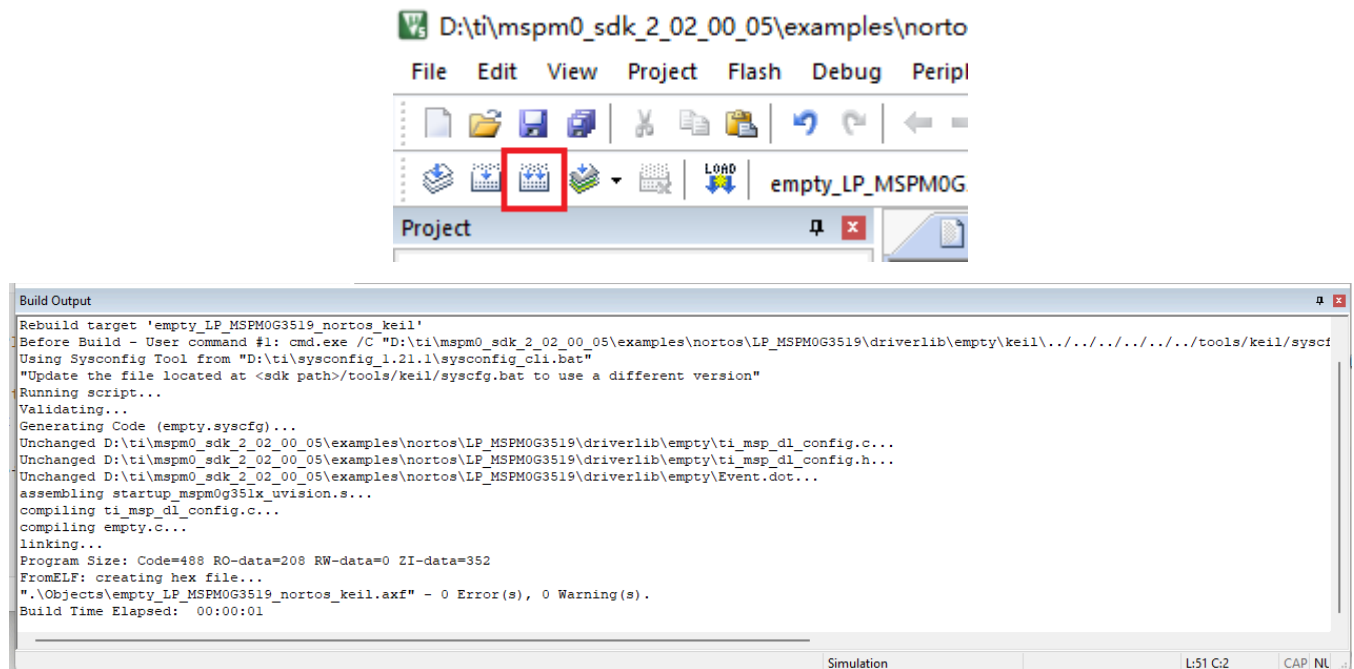
```

                                gCheckADC = true; // Set the flag position to 1
                                break;
default:
                                break;
        }
}

```

5. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.



4. Program Analysis

- empty.c

```

// Read ADC data
unsigned int adc_getValue(void)
{
    unsigned int gAdcResult = 0;

    // Software triggers ADC to start conversion
    DL_ADC12_startConversion(ADC_VOLTAGE_INST);
    // If the current state is converting, wait for the conversion to end
    while (false == gcheckADC) {
        __WFE();
    }
    // Get data
    gAdcResult = DL_ADC12_getMemResult(ADC_VOLTAGE_INST, ADC_VOLTAGE_ADCMEM_ADC_CH0);

    // Clear flag
    gcheckADC = false;

    return gAdcResult;
}

```

Call the `DL_ADC12_startConversion` function to start ADC conversion. If the ADC is still converting, the program enters low-power mode through `__WFE()` until the interrupt sets the `gCheckADC` flag, indicating conversion is complete. After conversion is complete, call `DL_ADC12_getMemResult` to get the conversion result and return.

```
// Tick timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

// ADC interrupt service function
void ADC_VOLTAGE_INST_IRQHandler(void)
{
    // Query and clear ADC interrupt
    switch (DL_ADC12_getPendingInterrupt(ADC_VOLTAGE_INST))
    {
        // Check whether data collection is completed
        case DL_ADC12_IIDX_MEM0_RESULT_LOADED:
            gCheckADC = true; // Set the flag position to 1
            break;

        default:
            break;
    }
}
```

SysTick_Handler: SysTick timer interrupt service function, used to implement delay. At each interrupt, `delay_times` decrements by 1 until it reaches 0 to stop the delay.

ADC_VOLTAGE_INST_IRQHandler: ADC interrupt service function. When ADC conversion is complete, check the ADC interrupt and set the flag `gCheckADC` to tell the main program data can be read.

```
int main(void)
{
    unsigned int adc_value = 0;
    unsigned int voltage_value = 0;

    SYSCFG_DL_init();

    // Clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    // Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
    // Enable ADC interrupt
    NVIC_EnableIRQ(ADC_VOLTAGE_INST_INT_IRQN);

    printf("adc Demo start\r\n");
    while (1)
    {
        // Get ADC data
    }
```

```

        adc_value = adc_getValue();
        printf("adc value:%d\r\n", adc_value);

        // Convert the data collected by ADC into voltage
        voltage_value = (int)((adc_value/4095.0*3.3)*100);

        printf("voltage value:%d.%d%d\r\n",
            voltage_value/100,
            voltage_value/10%10,
            voltage_value%10 );

        delay_ms(1000);
    }
}

```

After initializing the system configuration, enable serial port and ADC interrupts. In the main loop, get ADC data by calling `adc_getValue()`, convert it to voltage value and print.

5. Experimental Phenomena

After the program is downloaded, after configuring the serial assistant as shown in the figure below, adjust the potentiometer knob. You can see the voltage changing in real-time between 0-3.3v in the serial assistant receive window.

